AI Resource Counter

Feasibility, measurement evidence, and recommended architecture

Research date: 7 September 2026. Audience: the project owner and the implementation team. Status: research complete; extension implementation and live integration validation remain future work.

The proposed product is an open-source Chrome extension for ChatGPT, Claude, Gemini, and Perplexity. It should display conversation resource estimates inside each service and provide personal weekly/monthly totals and a company overview through a self-hosted server.

Decisions confirmed for this study

- Measure conversation operations. Exclude model training and hardware manufacturing from the headline totals.
- Cover text, visible thinking, web search, and uploaded documents. Identify portions that cannot be observed or estimated defensibly.
- Send pseudonymous numerical usage data to the company server. Do not send prompts or answers.
- Keep installation and everyday use simple for nontechnical employees.
- Publish the extension, backend, calculation logic, configuration schema, and supporting evidence openly.

Working boundary: data-centre operation and the electricity supply associated with it. Employee devices and internet transmission are excluded from initial resource totals and must be disclosed as exclusions. This boundary is a recommended interpretation of conversation use, not a claim that these other activities consume nothing.

1. Recommendation

Build the extension. The interface, local history, automatic synchronization, and company dashboard are feasible. The achievable measurement promise is: **the strongest available token evidence, plus transparent estimates of operational electricity, greenhouse-gas emissions, and water consumption.**

An extension cannot promise exact total input/output tokens or exact physical resource consumption for every request to all four consumer services. A tokenizer can count captured text, but it cannot reconstruct hidden context or unobserved model calls. Physical consumption additionally depends on the serving infrastructure. The original studies distinguish production instrumentation from estimates based on assumed hardware. Google production study, How Hungry is AI?.

Use a fresh Manifest V3 shell with a separate adapter for each website. Reuse selected extraction/UI code from Claude Counter, browser-compatible tokenizer libraries, and a pinned EcoLogits calculation component. None of the inspected extensions already meets the complete measurement requirement.

Two earlier conclusions need correction. Google provides official local Gemini tokenizers for explicitly supported model IDs. Anthropic published a historical tokenizer but explicitly warns that it is inaccurate for Claude 3 and later. The earlier ChatGPT Counter recommendation was also too generous: its documented unpacked code uses a character heuristic. Google tokenizer loader, Anthropic historical tokenizer, ChatGPT Counter implementation.

The product should help employees assess AI with evidence. It should not be designed to prove that every AI use is negligible. Show resource use and uncertainty consistently, including when a long reasoning or research task has a much larger footprint than a short answer. Avoid invented savings, offset claims, or rankings based on incomparable models.

2. Define what a token number means

There are four distinct levels of evidence. Store the level for every numerical field, rather than applying one accuracy badge to an entire conversation.

Evidence level	What the extension knows	Suitable user wording
Provider-reported usage	A validated response field reports a count for a defined request or component	Reported by provider
Official tokenizer, observed text	Captured text is counted with an explicitly mapped model tokenizer	Counted from visible content
Calibrated approximation	A substitute tokenizer or fitted correction is used	Estimated tokens
Missing information	Required content or a reliable count is unavailable	Not available / partly observed

“Counted from visible content” must not become “exact request usage.” DOM text can differ from original model output because Markdown, citation markers, tool blocks, and attachments have been transformed. Prefer original textual payloads when available. Keep tokenization accuracy separate from capture completeness.

New text is different from processed context

Illustrative example, excluding hidden instructions and caching:

Turn	New user tokens	Logical input if all prior messages are resent	New output tokens
1	100	100	200
2	50	350	150
Total	150	450	350

The conversation contains 500 unique visible tokens, but these two model requests process 800 logical input-plus-output tokens under the stated assumption. A conversation-size counter and a consumption counter answer different questions. The actual provider can truncate, summarize, retrieve, or reuse cached context; a browser reconstruction is therefore only a scenario when those decisions are hidden.

Store new input, observed/reconstructed context input, visible answer output, visible thinking text, provider-reported reasoning, and cache information separately. Provider-specific normalization must identify which counts are subsets. Adding cached input or reasoning to a total that already includes them would count them twice.

Thinking, search, and documents

Visible thinking can be a summary. Its length does not reveal the length of internal reasoning. OpenAI documents invisible reasoning tokens, Google distinguishes thought summaries from full thought-token use, and Anthropic documents summarized thinking. Tokenize a displayed summary as observed text; use a separate numeric reasoning count only if it is actually supplied. Otherwise record hidden reasoning as unknown. OpenAI reasoning, Gemini thinking, Claude extended thinking.

Search may cause query rewriting, retrieval, ranking, several model calls, and a final answer. A single user message is not necessarily one inference. Count observable calls and retrieved text, but report uncovered retrieval/ranking work separately. Do not attach an invented universal “search multiplier.”

For documents, local extraction can count available text. It cannot establish which pages, images, OCR results, or retrieved chunks the provider actually processed. An uploaded PDF's byte size or page count is not an exact token count. Google's local tokenizer is text-only; provider counting APIs support a different and broader request boundary. Google local tokenizer source, Anthropic counting documentation.

3. Tokenizer choices by provider

There is no evidenced universal “closest tokenizer.” Use a model registry with explicit mappings and fallbacks. The website name alone is insufficient.

Service/model evidence	Recommended implementation	Accuracy boundary
OpenAI's published GPT-5, GPT-4.1, GPT-4o, o1/o3/o4-mini mappings	o200k_base through a tested browser implementation	Reproducible observed-text tokenization for mapped models
Older GPT-4 / GPT-3.5 mappings	cl100k_base when explicitly applicable	Same boundary; do not apply to unknown models
Gemini models supported by Google's LocalTokenizer	Port the official model mapping and relevant tokenizer assets to a browser worker	Official mapping, but browser parity and consumer model identification still require verification
Modern Claude	Calibrated substitute; start by comparing o200k_base and Anthropic's historical tokenizer	Approximation; no validated current official offline tokenizer found
Perplexity using an identified third-party model	Select that model's verified tokenizer	Does not cover additional hidden Perplexity calls
Sonar or unknown/automatic routes	Test Llama-family and other candidates against available reference counts	No verified current Sonar tokenizer artifact found

OpenAI's official mapping and encoding assets are public. Its current file does not establish a mapping for every newer or internal model identifier; unsupported identifiers should stay explicitly unverified. The GPT-OSS mapping uses o200k_harmony and should not be confused with the GPT chat mapping. Official model map, encoding assets and integrity hashes.

For a TypeScript extension, **gpt-tokenizer** is a practical MIT-licensed component: it implements the relevant OpenAI encodings in JavaScript. Bundle the specific encodings and run them in a worker; verify against official Python tiktoken fixtures. Library claims about compatibility do not establish consumer-web request totals. gpt-tokenizer repository, tiktoken license.

Google's inspected loader maps Gemini 2.0/2.5 and selected Gemini 3 previews to a pinned Gemma3 SentencePiece artifact. It maps selected Gemini 3.1 variants and Gemini 3.5 Flash to a Gemma4 tokenizer. This is an explicit Google mapping, not an inference that Gemma and Gemini must share tokenizers. Pin the exact assets, notices, model list, and hashes at implementation time; newer model names may be unsupported. Official loader and mappings, Google local counting examples.

For Claude, Anthropic's old MIT tokenizer is a benchmark candidate, not an exact modern solution. The current counting documentation describes a newer tokenizer for Claude 4.7 and later and warns that counts may differ from actual inference. Its reported change relative to earlier Claude tokenizers is not a correction factor for an OpenAI tokenizer. Historical tokenizer warning, current counting behavior.

Perplexity documents multiple third-party model providers. Historical descriptions of Sonar's Llama basis differ across official materials; neither establishes an exact current tokenizer mapping. Its public API has token-usage fields, but that is not proof that the consumer website returns those fields. Perplexity model disclosure, Sonar API usage schema.

How to choose the closest approximation honestly

Build a public/synthetic evaluation corpus before selecting a Claude or Sonar fallback. Include English and German prose, mixed languages, code, numbers, tables, URLs, Unicode, long reports, and document excerpts. Use short and long examples and hold back a test set from any calibration.

During development, compare candidates with official model-specific counting endpoints and available inference usage using the same explicit model version. Separate content tokens from request-envelope overhead. Record exact-match rate, signed bias, median absolute percentage error, high-percentile error, and interval coverage, broken down by language and content type. Handle zero-token cases separately.

Select the fallback from measured results. Until those results exist, o200k_base is a convenient provisional approximation for Claude, not a proven winner. Do not advertise a numerical accuracy percentage from this research: no such benchmark was run.

Calibration can use developer-owned credentials and synthetic inputs. Employees need no API keys, and their text need not leave their browser for calibration. Calling a counting API adds another network operation and does not reveal the original consumer request's hidden context, so it should not be the everyday metering mechanism.

4. Open-source projects worth reusing

The following is a read-only source audit, not an installation or live compatibility test. License statements apply to the inspected repositories, not automatically to every bundled asset. Pin a commit and preserve its notices before incorporating code.

Project	Evidence from the inspected code/license	Decision
Claude Counter	MIT; Claude conversation capture and embedded UI; real o200k_base applied as a substitute; omits thinking and document/image blocks	Best focused donor for Claude extraction/UI; replace consumption accounting
AI Wattch	Repository declares MIT; ChatGPT, Claude, Gemini; character-based estimates and DOM message capture	Possible sustainability UI donor; replace counting, add Perplexity, inspect logging
Alba	MIT declared; all four websites and an embedded interface; generic coefficients and heuristic tokens	Appearance/reference donor; replace accounting and remove remote prompt optimization
ChatGPT Counter, Dhruvg0	MIT declared; documented unpacked folder uses characters/4 despite README tokenizer claims	Do not use as the accuracy foundation
TokenLens	Declares ISC; parsers look for public-API-style usage fields; heuristic fallback	Parser architecture reference; consumer compatibility not established
AIRadar	License explicitly restricts reuse and says it is not open source	Unsuitable for the intended unrestricted project
EcoPrompt, IB2M	Noncommercial CC BY-NC-SA terms; fixed query-tier factors	Unsuitable unrestricted base; no exact tokenizer foundation
EcoLogits	MPL-2.0 Python calculation library; separate API project uses AGPL-3.0	Reuse the core methodology component; it is not a browser capture extension

Sources: Claude Counter code and MIT license; AI Wattch estimator; Alba configuration; ChatGPT Counter code; TokenLens interceptor; AIRadar license; EcoPrompt license; EcoLogits core, EcoLogits API.

Three source findings matter particularly:

- Claude Counter walks the active conversation branch and adds its observable contents. That measures context size, not cumulative inference input across turns. It explicitly excludes thinking and document/image blocks, although extracted attachment text and some tool payloads are included. Its cache countdown is not evidence of a provider cache hit. Token accounting source.
- TokenLens's interceptors look for fields such as API-style usage objects. Its Gemini matcher targets the developer API host. A parser that knows a field name does not prove that Gemini's consumer service supplies that field. Interceptor source.

- Alba's capture code processes existing messages and uses a fixed delay for responses, creating risks of recounting history or missing the end of a stream. Its configuration enables a remote optimizer when configured; that feature sends prompt content and conflicts with the proposed data policy. Alba content script, configuration.

Recommendation: start a clean WXT/TypeScript Manifest V3 project and bring in narrow, attributed components. WXT is MIT-licensed and supports MV3 and TypeScript. Replacing most of a fork's capture, accounting, synchronization, and configuration logic would reduce the benefit of forking the whole extension. WXT project.

5. Offset AI: useful concept, calculations need revision

Offset AI's published method multiplies prompt counts by service-specific constants. It acknowledges prompt and model variability. Its homepage advertises four-site tracking and team reporting. No reusable public source/license was found. Methodology, product.

The methodology's benchmark citation opens an unrelated bovine-embryo paper. The actual How Hungry is AI? study estimates energy using performance data and inferred hardware; its inspected version does not contain Perplexity or Sonar. This does not substantiate Offset AI's claimed Perplexity retrieval benchmark. Linked paper, actual study.

Its formula adds PUE to Google's 0.24 Wh figure, although Google's figure already includes facility overhead. Its on-site water calculation also applies PUE to a WUE term without establishing a matching denominator. These are reasons to implement a boundary-aware calculation rather than copy the constants. Google measurement boundary, water accounting framework.

6. What the environmental evidence supports

Evidence	What it tells us	Appropriate use
Google's May 2025 Gemini Apps study	Reported median 0.24 Wh and 0.26 mL direct cooling water; 0.03 g CO2e combines market-based electricity accounting with additional embodied/direct contributions	Historical production reference; not a universal per-message or present-day factor
Sam Altman's June 2025 disclosure	Average ChatGPT query stated as 0.34 Wh and 0.000085 US gallons, approximately 0.32 mL	Coarse reference; no sufficient workload/boundary details for precise model calibration
ML.ENERGY benchmark	Public measurements across models, tasks, hardware and serving configurations	Empirical foundation for surrogate models; benchmark hardware is not proven to be the commercial service's hardware
EcoLogits	Open calculation engine and model/environment assumptions, with usage and embodied components separable	Auditable starting point for resource estimates
Li et al., Making AI Less Thirsty	Separates direct cooling water from water consumed in electricity generation	Correct water-accounting framework; old GPT-3 examples are not current product constants
How Hungry is AI?	Estimates closed-model energy with inferred hardware and performance data	Sensitivity/comparison evidence; not direct provider power-meter readings

Sources: Google study, Altman disclosure, ML.ENERGY data, EcoLogits methodology, Li et al., Jegham et al..

These figures are not directly interchangeable. A median is not the arithmetic mean needed to infer aggregate consumption from a request count. Fleet averages also do not establish the consumption of a specific long-context or reasoning request. Multiplying a historical median by company prompts should be labeled a reference scenario, not a measured total.

Recommended energy model

Use the best evidence applicable to the observed model and workload, in this order:

1. Provider-supplied operational measurements with a documented boundary, if available for the relevant workload.
2. An empirical model covering the relevant input length, output length, architecture, and serving configuration.
3. An explicitly identified surrogate model with scenario ranges for unknown deployment parameters.
4. A historical per-query reference only when stronger evidence is absent, visibly labeled as a fallback.

The first implementation can use EcoLogits as a reference engine while building a better input-aware layer. Its current source calculates GPU energy primarily from output tokens, active parameters, and batch assumptions. It has no explicit input-token argument in the inspected LLM calculation interface. Therefore it cannot resolve the user's long-document requirement by itself. Use its operational usage component rather than its combined usage-plus-embodied total. Calculation implementation.

EcoLogits's published defaults assume particular GPU hardware and deployment behavior. Its proprietary-model sizes are estimates. Google's TPU deployment and unknown commercial batching can depart from that surrogate. Treat the engine as an auditable estimate whose fit must be assessed, not a physical meter. Method assumptions, proprietary-model assumptions.

For higher accuracy, model prompt processing and token generation separately. TokenPowerBench explicitly separates these phases; newer measured studies show request energy depends on batch size and context, not just the sum of input and output tokens. These are useful development datasets, but transferring an open-model benchmark to a proprietary service remains an inference. TokenPowerBench, request/token energy characterization, ML.ENERGY v3 analysis.

A suitable proposed structure is:

```
E_IT = E_prefill(observed context, cache scenario, model, hardware)
     + E_decode(generated tokens including known reasoning, context, batching)
     + allocated host and idle energy within the chosen boundary
     + separately estimated tool-operation energy, when supported
```

This is a design structure, not a calibrated formula. Each term needs evidence. Do not invent a fixed price-to-energy conversion, infer GPU energy from browser waiting time, or multiply all reasoning requests by an arbitrary constant. Browser latency includes queuing, networking, search, and pauses. Cached inputs can still require work; an API price discount is not a measured energy discount.

7. Equations and uncertainty

Define E_IT in kWh as allocated server energy before facility overhead, and E_facility as energy including that overhead. Let C be the electricity emissions factor in g CO2e/kWh; WUE_IT be direct water consumption in L/kWh of IT electricity; and W_elec be upstream water consumption in L/kWh of supplied electricity.

```
E_facility [kWh] = E_IT [kWh] * PUE
CO2e_electricity [g] = E_facility [kWh] * C [g/kWh]
Water_direct [L] = E_IT [kWh] * WUE_IT [L/kWh_IT]
Water_upstream [L] = E_facility [kWh] * W_elec [L/kWh]
Water_total [L] = Water_direct + Water_upstream
```

The water terms intentionally use different energy denominators. If a published WUE instead uses facility electricity, apply that denominator explicitly. If the energy source already includes PUE, do not multiply by PUE again. Li et al. express the operational water calculation as IT energy multiplied by direct WUE plus PUE-weighted upstream WUE. Original water framework.

For the initial carbon display, label this “estimated electricity-related operational CO2e.” Direct combustion associated with backup generation is additional operational impact unless already included in the factor; add it only with compatible evidence. Do not claim an all-inclusive carbon inventory when this component is missing.

Illustration only, using invented demonstration parameters: E_IT = 0.50 Wh, PUE = 1.20, C = 400 g/kWh, WUE_IT = 0.50 L/kWh, and W_elec = 2.0 L/kWh. The resulting facility electricity is 0.60 Wh, electricity emissions are 0.24 g CO2e, direct water is 0.25 mL, upstream water is 1.20 mL, and total water is 1.45 mL. These are arithmetic fixtures, not proposed product defaults.

Rules for selecting environmental factors

- Use the serving region if there is reliable evidence for it. An employee's location, browser IP, CDN edge, account billing country, or the company's server location does not establish the inference location.
- When location is unknown, expose that uncertainty and use a documented fleet/world scenario or a range. Do not silently hardcode Germany because employees work there, or the United States because a provider is American.
- Keep location-based and market-based carbon estimates separate. Renewable-energy procurement claims do not imply zero physical resource use. The Google study explicitly distinguishes the two methods. Google accounting definitions.
- Distinguish water consumption from withdrawal. Show direct cooling and electricity-supply water separately in details. Litres alone do not measure local water scarcity.
- Store the factor's year, source, geography, units, accounting boundary, and license. Preserve old calculation versions so a changed coefficient does not silently rewrite last month's report.

Report ranges without inventing confidence

Maintain uncertainty in capture completeness, tokenizer substitution, hidden context, model identification, hardware/utilization, location, and environmental factors. Missing data must be null/unknown, not zero.

Show a central scenario and an explanatory range only where scenarios have evidence. Call it an “estimated range,” not a 95% confidence interval unless calibrated statistical coverage supports that interpretation. If hidden calls have no defensible finite bound, report a partial estimate with an unquantified component instead of manufacturing a reassuring upper bound.

Aggregation does not remove shared errors. If all company requests use an incorrect model-size assumption, collecting more requests does not average that error away. Aggregate coherent low/central/high scenarios across a period; retain shared model and region uncertainty. Do not simply divide uncertainty by the square root of the number of requests.

8. Extension design

Suggested components: WXT + TypeScript, four provider adapters, a tokenizer worker, a versioned estimator, an extension-owned local database, and a background synchronization queue. This is a proposed architecture, not an implementation delivered by this study.

```
Website request/response and visible conversation
  -> Provider adapter and completeness flags
  -> Local tokenizer + normalized usage event
  -> Local event ledger + resource calculation
  -> In-chat bar and personal period totals
  -> Pseudonymous numerical upload queue
  -> Company HTTPS server -> aggregate dashboard
```

Capture mechanism

Chrome supports content scripts, early injection, and MAIN-world execution. A narrowly scoped MAIN-world bridge can observe fetch/XHR responses that the page receives, while an isolated content script provides UI and communicates with the extension. Chrome's webRequest API alone is insufficient: its public surface has no general response-body retrieval method and does not expose WebSocket message contents. Content scripts, webRequest.

Claude Counter supplies an example of cloning fetch responses and parsing conversation/stream events. This is feasibility evidence for the approach, not a guarantee that current versions of all four sites use equivalent protocols. Other transports and worker-originated requests may need different adapters or remain partial. Claude capture bridge.

The bridge should observe the original traffic without replaying prompts or consuming the stream needed by the page. Bound stream buffering and preserve cancellation. DOM inspection is a fallback for visible messages and UI state, with a lower completeness classification. Never search arbitrary JSON recursively for a number named tokens and immediately declare it request usage; require a validated provider schema and semantics.

Because MAIN-world data shares the page's trust boundary, validate all bridge messages before the extension processes them. Permit only known event types, bounded payloads, the expected site, and sensible numerical values. Keep server credentials and enrollment tokens out of the page context. Client-generated totals cannot become tamper-proof audit records merely by adding authentication.

A consumption ledger, not a page counter

Use a stable event ID for each observed generation attempt and revision. Distinguish request start, provisional output, completion, cancellation, failure, regeneration, and continuation. Deduplicate repeated stream events and updates. Count genuine regenerations separately even when the final wording is identical.

Opening an old conversation, scrolling history into view, switching tabs, or revisiting a message must not add new consumption. Preserve consumed work from abandoned branches. Treat existing conversations at installation as history, not new usage; an optional historical import would need its own separate estimate category.

Token counts for chunks are not generally additive because token boundaries can span chunks. Re-tokenize the completed text or use a validated incremental tokenizer. Replace provisional estimates with final values using versioned upserts rather than accumulating every provisional value. Preserve partial generated output after cancellation and flag any unobserved server work.

Use extension-owned IndexedDB for the event ledger and chrome.storage for preferences. Ordinary page localStorage is the website's storage, and MV3 background workers can stop between tasks; the queue must survive restarts. Extension storage guidance, Chrome storage API.

9. Self-hosted company service

Recommend a small HTTPS application with a relational database, an authenticated ingestion endpoint, and read-only aggregate endpoints. A Python/FastAPI service can reuse EcoLogits directly. A TypeScript browser implementation of the selected formulas can supply immediate and offline estimates; parity fixtures must verify that browser and server calculate the same results. If that port is deferred, clearly show pending resource calculations while offline.

Use one pinned methodology bundle on both sides. The server should validate and recompute estimates from permitted numerical features where possible, retaining the original method version. It must not trust arbitrary uploaded resource totals. Keep enough versioned numerical features for reproducibility without retaining text.

Server field group	Suggested contents	Excluded contents
Identity	Tenant, random installation/user pseudonym, authenticated device, event ID	Employee name/email unless a later explicit feature requires it
Workload	Provider, observed model, token categories, counts of observed search/document events	Prompts, responses, thought text, retrieved passages
Coverage	Capture status, tokenizer provenance, unknown-component flags	Raw response bodies or network logs
Estimate	Energy, CO2e, direct/upstream water, scenario range, method version	Invented exact totals for missing operations
Period	Coarse event time/date with company timezone and synchronization revision	Conversation titles, full URLs, file names

Keep raw conversation IDs local; the server can use random event IDs for idempotency. Avoid uploading unsalted text hashes, which can reveal known phrases through guessing. Use numerical document features only when they improve a supported estimate, and do not log content accidentally in errors or debugging output.

For first deployment, use Docker Compose with the application and PostgreSQL, migrations, backups, and a documented TLS reverse proxy. A company configures its own domain and retains its own data. No particular VPS has been selected or changed in this research.

Enrollment and automatic synchronization

An arbitrary server URL alone cannot safely establish company membership. Provide a company enrollment link or a short-lived invitation that installs configuration and exchanges for a scoped device credential. The user experience can still be: install, connect to company once, and then use AI normally.

Managed Chrome installations can preload the server URL, tenant, and approved policy through chrome.storage.managed. This supports an install-and-use experience where IT configures the extension centrally. Managed storage schema.

For unmanaged installations, request permission for the chosen HTTPS server origin during enrollment. Chrome supports optional host permissions; declaring optional patterns does not grant access to all matching hosts automatically. Production should use a valid HTTPS name rather than a bare IP with certificate warnings. Permission declarations, runtime permissions.

Queue numerical events locally, batch uploads, retry with backoff, and use authenticated idempotent upserts. Track acknowledgments and queue age. A network outage must neither lose usage nor block the chat. Use company timezone for weekly/monthly boundaries while retaining an unambiguous time basis.

The monthly overlay should open once when an employee next uses a supported site, or on demand. The extension cannot guarantee a popup at a particular time when Chrome is closed. The dashboard should say “tracked company usage” and show reporting coverage, last synchronization, and unestimated events. It does not measure employees without the extension or usage in unsupported apps.

Employee trust and distribution

Provide a short disclosure before enrollment, a visible connected-company indicator, pause/disconnect controls consistent with company policy, retention settings, deletion/export, and aggregate-only access for ordinary users. Suppress small group breakdowns and avoid employee rankings. Pseudonymization should not be described as anonymity: reidentifiable data can still be personal data. GDPR text, Article 4 and Recital 26.

Chrome Web Store disclosures must accurately describe capture and server transmission. Package executable JavaScript and WASM with the extension; remote configuration should be constrained data, not remotely downloaded code. Chrome user-data policy, remote-code policy.

Publish through the Chrome Web Store for ordinary installation. An unpacked repository is appropriate for developers but not the intended employee experience. Enterprise deployment can use Chrome's supported managed installation paths. Chrome installation guidance.

10. Interface that belongs in each service

The default view should be a compact bar beneath the composer:

```
Conversation | Tokens in / out | Estimated Wh | g CO2e | mL water | Details
```

During drafting, label tokens as a preview; do not add draft text to consumption. During generation, show a provisional state. After completion, display the final observed count or estimate. A small approximation symbol and one consistent “Estimated resource use” label are enough in the bar; the details panel can carry the richer explanation.

Use a shared accessible component with provider-specific theme adapters. Read the site's effective background, text, border, and accent styling, and handle light/dark mode changes. Adapt spacing and typography to the host. A Shadow

DOM container can limit style collisions, with selected theme values passed into it. Avoid hardcoding one current screenshot's colours as a permanent provider palette.

Use an anchored composer container, not a fixed viewport coordinate. Test narrow windows, zoom, collapsed sidebars, long drafts, attachments, and overlapping send/tool buttons. Keyboard users must be able to open details, navigate the overlay, close it with Escape, and return focus to the chat.

The details overlay needs only three destinations: this conversation, my week/month, and company week/month. Show energy, carbon, and water first, then a short explanation of the estimate. Place direct/upstream water, input/context distinctions, data quality, methodology version, sources, and synchronization health behind a single details expansion.

Use at most one understandable comparison, with correct units and a cited assumption. For example, a bulb comparison requires an explicit wattage and duration. Do not claim emissions were saved by using AI unless the displaced activity and counterfactual have actually been evaluated.

11. Validation before employee release

The next step should be a measurement prototype, before investing heavily in the dashboard. Its deliverable is a supported-site/model/mode matrix backed by fixtures.

Capture verification

Use synthetic conversations in each website and record the request/response schemas locally in a controlled development environment. Confirm what actually arrives for normal chat, thinking, search, document upload, regenerate, stop, edit/branch, temporary chat, and auto model routing. Redact session credentials before retaining fixtures.

Acceptance: every emitted usage field has defined semantics and evidence. Missing fields remain unknown. A website or protocol change produces an adapter-health warning instead of a plausible-looking zero. No claim of a stable private endpoint contract is made without observed evidence.

Tokenizer verification

For mapped OpenAI/Gemini models, require parity with official reference implementations on the supported text corpus. Separately measure capture completeness. For Claude/Sonar substitutes, publish held-out bias/error results by model and language. Do not hide a failure on German text behind an English-only average.

Accounting verification

Use fixtures for reloading history, several open tabs, stream boundaries, interrupted responses, retries, duplicate upload batches, database restarts, clock boundaries, and regenerated answers. Acceptance: refreshing or scrolling adds no events; a real extra generation does; repeat uploads leave totals unchanged; a newer final estimate replaces its provisional revision.

Resource verification

Test all unit conversions, single application of PUE, WUE denominators, direct versus upstream water, usage versus embodied results, and input/output/reasoning subset rules. Compare browser and server outputs from the same method bundle. Evaluate short chat and long-document workloads separately.

Provider disclosures are plausibility anchors, not a per-request validation oracle. Actual energy accuracy for proprietary deployments cannot be certified without matching infrastructure telemetry. Any surrogate's uncertainty must survive into company totals.

Privacy and user experience

Inspect outgoing extension traffic and database rows using synthetic distinctive text. Acceptance: no prompt, response, thought content, filename, conversation title, full chat URL, or content hash appears centrally or in telemetry logs. Validate tenant isolation, credential revocation, and the disclosure/enrollment flow.

Test each site in both themes, at common zoom levels, using keyboard navigation and a small nontechnical employee pilot. Measure whether users understand that resource figures are estimates and whether they can explain the three headline units. Optimize clarity before adding more metrics.

12. Licensing, maintenance, and release sequence

Use a clear open-source license from the beginning, even if publication occurs later. A permissive license such as Apache-2.0 is a reasonable default for newly authored code; preserved third-party components keep their applicable obligations. EcoLogits core is MPL-2.0, while its existing API server is AGPL-3.0, so choose deliberately whether to depend on the core or fork that server. Noncommercial and source-available projects should not be represented as unrestricted open-source dependencies. EcoLogits, EcoLogits API, AIRadar license.

Publish a dependency/license inventory, pinned builds, model/tokenizer registry, methodology changelog, calculation fixtures, schema versioning, and a self-hosting guide. Pin tokenizer assets separately from library versions. Environmental datasets can have different redistribution terms from the calculation code; record those terms before packaging data.

Recommended release sequence:

1. **Measurement prototype:** establish capture capability and tokenizer parity across the four sites; publish the limitations matrix.
2. **Local beta:** stable event ledger, consistent in-chat bar, personal period totals, and operational estimates with documented exclusions.
3. **Company pilot:** self-hosted ingestion/database, one-time enrollment, aggregate overlay, offline synchronization, and a small employee usability/privacy test.
4. **Public release:** store packaging, source publication, reproducible deployment, documentation, and a maintained adapter test suite.

Do not promote a site/model combination from experimental until its capture and counting evidence passes. Site support and estimation support are separate: a site can have a working bar while a newly released model's resource profile remains unavailable.

13. Remaining decisions and research limits

No additional access is required to act on this research recommendation. Implementation will require a controlled way to test each consumer site, and synthetic counting references for the models selected for launch. The intended company server, deployment environment, branding, and retention duration can be chosen during implementation; this report does not select or modify infrastructure.

The work reviewed official SDK source, primary measurement/modeling studies, Chrome documentation, and extension implementations. It independently checked the most consequential tokenizer, formula, and reuse claims. It did not install the candidate extensions, capture new live consumer conversations, perform paid API benchmarks, measure data-centre power, or audit an entire production deployment.

The earlier Claude.ai session found no exact usage fields in the responses inspected then. That is a limited observation and cannot prove absence across every model, plan, endpoint, or future site version. Public developer API schemas also do not establish what the consumer browser receives.

Research stopped when the major feasibility decisions had primary support and the remaining questions required implementation experiments rather than more general searching. The decisive next artifact is a capture-and-calibration matrix, followed by the extension described here. An exact meter for all cloud computation is not currently substantiated; a useful, reproducible, privacy-conscious estimator is.